Salesforce Portfolio Projects: Implementation Guide

Program Waitlist Manager + Agentforce Enrollment Assistant

Overview

Both projects live in one Agentforce-enabled Developer Edition org and one GitHub repo. Project 2's agent reuses Project 1's data model and Apex, so the portfolio tells one story: a childcare nonprofit's enrollment system with an AI assistant on top.

The story: a nonprofit runs programs (for example Pre-K Morning, Infant Care) with fixed seat capacity. Families enroll, overflow goes to a waitlist, and freed seats are offered to the next family with a time-limited offer. Program coordinators then use an Agentforce agent to query waitlists, find openings, and act on enrollments in plain language.

What you received

File	What it is
`enrollment-portfolio-sfdx.zip`	The full SFDX project (source format). Unzip, open in VS Code, push to GitHub, deploy with the CLI.
`enrollment-portfolio-deploy.zip`	Metadata API format with `package.xml` at the root. Deploy directly through Workbench, no CLI needed.
`Implementation_Guide.pdf`	This document.

Every code listing in this guide is the exact file shipped in the zips.

Part 0: Deploy to your org

0.1 Get the right org

Sign up for the free Salesforce Developer Edition that includes Agentforce and Data Cloud (announced on the Salesforce Developers blog in March 2025; it stays active as long as you keep using it). An older standard Developer Edition may not have Agentforce, which Project 2 needs. Build both projects in this one org.

0.2 Option A: Salesforce CLI (recommended)

```
unzip enrollment-portfolio-sfdx.zip && cd enrollment-portfolio
sf org login web --alias portfolio --set-default
sf project deploy start --manifest manifest/package.xml --test-level RunLocalTests
sf org assign permset --name Enrollment_Coordinator
sf apex run --file scripts/apex/seed.apex
sf apex run --file scripts/apex/schedule-expiry.apex
sf apex run test --code-coverage --result-format human --wait 10
```

`RunLocalTests` proves the whole suite passes during deployment; drop the flag if you want a faster deploy.

0.3 Option B: Workbench (no CLI)

1. Go to workbench.developerforce.com and log in to your org.
2. Migration, Deploy. Choose `enrollment-portfolio-deploy.zip`.
3. Tick **Single Package** and **Rollback On Error**, then Next and Deploy.
4. In Setup, Permission Sets, assign **Enrollment Coordinator** to yourself.
5. Open the Developer Console, Debug, Open Execute Anonymous Window. Paste and run `scripts/apex/seed.apex`, then `scripts/apex/schedule-expiry.apex` (both are in the SFDX zip).

0.4 What deploys, and what you configure by hand

Deploys automatically	Configure in the UI (about 30 minutes)
Program__c, Enrollment__c, Enrollment_Change__e	Add the LWC to the Program record page (step 3c)
All Apex classes, triggers, tests	Notification Flow (step 3d)
programEnrollment LWC	Draft Family Update prompt template (Project 2, step 2d)
Enrollment_Coordinator permission set, Program tab	Enrollment Assistant agent, topic and actions (Project 2, step 3)

The right-hand items are left manual on purpose: their metadata varies by org and release, and a failure in any one of them would roll back the entire deployment. Building them by hand is also what interviewers want to hear you describe.

0.5 Repo and demo video

1. Push the unzipped project to a public GitHub repo. The included `README.md` is ready to go.
2. After changes in the org, pull them back with `sf project retrieve start --manifest manifest/package.xml` and commit.
3. Record a 2 to 3 minute demo video for each project. Interviewers rarely log into your org; they skim the README and watch the video.

Project 1: Program Waitlist Manager

Step 1: Data model

Two custom objects and one platform event carry the whole feature. Custom objects keep the project portable to any Developer Edition org; in interviews, explain how they map to Nonprofit Cloud's Program and Program Enrollment objects.

Program__c

Field	Type	Notes
Name	Text(80)	e.g. Pre-K Morning
Capacity__c	Number(4,0), required	Seats available
Offer_Window_Hours__c	Number(3,0), default 48	Time a family has to accept
Start_Date__c	Date	
Seats_Held__c	Roll-up COUNT of Enrolled + Offered	Display only; Apex re-counts under a lock
Seats_Remaining__c	Formula: Capacity - Seats Held	Drives the capacity bar

Enrollment__c (Name is an auto number, ENR-00001)

Field	Type	Notes
Program__c	Master-Detail(Program__c)	Enables the roll-up
Contact__c	Lookup(Contact), required	The family contact
Status__c	Restricted picklist	Enrolled, Waitlisted, Offered, Declined, Expired, Withdrawn
Waitlisted_At__c	Date/Time	Queue order, oldest first
Offered_At__c, Offer_Expires_At__c, Enrolled_At__c	Date/Time	Audit trail

Enrollment_Change__e (platform event, Publish After Commit): Enrollment_Id__c, Program_Id__c, Contact_Id__c, New_Status__c.

Design note: queue order uses `Waitlisted_At__c` (with the auto-number Name as a tie-breaker) instead of a stored position number, so withdrawals never require renumbering the waitlist. Position is computed at read time.

Step 2: Apex service layer and waitlist logic

All enrollment rules live in one service class; the trigger and LWC only call it. A seat frees up whenever an Enrolled or Offered record moves to any other status, and the trigger immediately offers that seat to the oldest waitlisted family.

Status flow

From	Event	To
New request	Seat open	Enrolled
New request	Program full	Waitlisted
Waitlisted	Seat freed (oldest first)	Offered
Offered	Family accepts	Enrolled
Offered	Family declines / offer times out	Declined / Expired (frees the seat)
Enrolled or Waitlisted	Family withdraws	Withdrawn (frees the seat if it held one)

Every time a seat is freed, the trigger runs `fillOpenSeats()` and the next family moves from Waitlisted to Offered. That automatic promotion is the key behavior to demo.

2a. EnrollmentService.cls

File: `force-app/main/default/classes/EnrollmentService.cls`

```
/**
 * All enrollment rules live here. The trigger, LWC controller, batch job
 * and Agentforce actions call these methods and nothing else writes status.
 */
public with sharing class EnrollmentService {
    public class EnrollmentException extends Exception {}

    public static final Set<String> SEAT_HOLDING = new Set<String>{ 'Enrolled', 'Offered' };
    public static final Set<String> ACTIVE = new Set<String>{ 'Enrolled', 'Offered', 'Waitlisted' };

    // Allowed status moves; anything else is rejected
    private static final Map<String, Set<String>> TRANSITIONS = new Map<String, Set<String>>{
        'Waitlisted' => new Set<String>{ 'Withdrawn' },
        'Offered'    => new Set<String>{ 'Enrolled', 'Declined', 'Expired' },
        'Enrolled'   => new Set<String>{ 'Withdrawn' }
    };

    public static Enrollment__c enroll(Id programId, Id contactId) {
        // Lock the program row so two coordinators cannot take the last seat at once
        Program__c program = [SELECT Id, Capacity__c FROM Program__c WHERE Id = :programId FOR UPDATE];

        Integer duplicates = [SELECT COUNT() FROM Enrollment__c
                              WHERE Program__c = :programId AND Contact__c = :contactId
                              AND Status__c IN :ACTIVE];
        if (duplicates > 0) {
            throw new EnrollmentException('This contact already has an active enrollment in this program.');
        }

        Integer held = [SELECT COUNT() FROM Enrollment__c
                        WHERE Program__c = :programId AND Status__c IN :SEAT_HOLDING];

        Enrollment__c e = new Enrollment__c(Program__c = programId, Contact__c = contactId);
        if (held < program.Capacity__c) {
            e.Status__c = 'Enrolled';
            e.Enrolled_At__c = System.now();
        } else {
            e.Status__c = 'Waitlisted';
            e.Waitlisted_At__c = System.now();
        }
        insert e;
        return e;
    }

    public static void changeStatus(Id enrollmentId, String newStatus) {
        Enrollment__c e = [SELECT Id, Status__c FROM Enrollment__c WHERE Id = :enrollmentId FOR UPDATE];
        Set<String> allowed = TRANSITIONS.get(e.Status__c);
        if (allowed == null || !allowed.contains(newStatus)) {
            throw new EnrollmentException('Cannot move an enrollment from ' + e.Status__c + ' to ' + newStatus + '.');
        }
        e.Status__c = newStatus;
        if (newStatus == 'Enrolled') {
            e.Enrolled_At__c = System.now();
        }
        update e; // the trigger handles any freed seat
    }

    public static void fillOpenSeats(Set<Id> programIds) {
        // SOQL does not allow ORDER BY in a locking query, so lock programs first
        Map<Id, Program__c> programs = new Map<Id, Program__c>([
            SELECT Id, Capacity__c, Offer_Window_Hours__c FROM Program__c
            WHERE Id IN :programIds FOR UPDATE
        ]);

        Map<Id, Integer> held = new Map<Id, Integer>();
        for (AggregateResult ar : [SELECT Program__c programId, COUNT(Id) total FROM Enrollment__c
                                   WHERE Program__c IN :programIds AND Status__c IN :SEAT_HOLDING
                                   GROUP BY Program__c]) {
            held.put((Id) ar.get('programId'), (Integer) ar.get('total'));
        }

        Datetime now = System.now();
        List<Enrollment__c> offers = new List<Enrollment__c>();
        // Name (auto number) breaks ties when several families joined in the same millisecond
        for (Enrollment__c w : [SELECT Id, Program__c FROM Enrollment__c
                                WHERE Program__c IN :programIds AND Status__c = 'Waitlisted'
                                ORDER BY Program__c, Waitlisted_At__c ASC, Name ASC]) {
            Program__c p = programs.get(w.Program__c);
            if (p == null) {
                continue;
            }
            Integer seatsHeld = held.containsKey(p.Id) ? held.get(p.Id) : 0;
            if (seatsHeld < p.Capacity__c) {
                Integer hours = p.Offer_Window_Hours__c == null ? 48 : p.Offer_Window_Hours__c.intValue();
                w.Status__c = 'Offered';
                w.Offered_At__c = now;
                w.Offer_Expires_At__c = now.addHours(hours);
                held.put(p.Id, seatsHeld + 1);
                offers.add(w);
```

```
            }
        }
        if (!offers.isEmpty()) {
            update offers;
        }
    }
}
```

2b. Trigger and handler

File: `force-app/main/default/triggers/EnrollmentTrigger.trigger`

```
trigger EnrollmentTrigger on Enrollment__c (after insert, after update, after delete) {
    EnrollmentTriggerHandler.run(Trigger.operationType, Trigger.new, Trigger.oldMap);
}
```

File: `force-app/main/default/classes/EnrollmentTriggerHandler.cls`

```
public with sharing class EnrollmentTriggerHandler {
    @TestVisible
    private static Integer eventsPublished = 0;

    public static void run(System.TriggerOperation op, List<Enrollment__c> newList, Map<Id, Enrollment__c> oldMap) {
        Set<Id> freed = new Set<Id>();
        List<Enrollment_Change__e> events = new List<Enrollment_Change__e>();

        switch on op {
            when AFTER_INSERT {
                for (Enrollment__c e : newList) {
                    events.add(toEvent(e));
                }
            }
            when AFTER_UPDATE {
                for (Enrollment__c e : newList) {
                    Enrollment__c old = oldMap.get(e.Id);
                    if (e.Status__c == old.Status__c) {
                        continue;
                    }
                    events.add(toEvent(e));
                    if (EnrollmentService.SEAT_HOLDING.contains(old.Status__c)
                        && !EnrollmentService.SEAT_HOLDING.contains(e.Status__c)) {
                        freed.add(e.Program__c);
                    }
                }
            }
            when AFTER_DELETE {
                for (Enrollment__c old : oldMap.values()) {
                    if (EnrollmentService.SEAT_HOLDING.contains(old.Status__c)) {
                        freed.add(old.Program__c);
                    }
                }
            }
        }
        // fillOpenSeats only moves Waitlisted to Offered, which never frees a seat,
        // so the nested trigger run publishes events and stops. No recursion guard needed.
        if (!freed.isEmpty()) {
            EnrollmentService.fillOpenSeats(freed);
        }
        if (!events.isEmpty()) {
            EventBus.publish(events);
            eventsPublished += events.size();
        }
    }

    private static Enrollment_Change__e toEvent(Enrollment__c e) {
        return new Enrollment_Change__e(
            Enrollment_Id__c = e.Id,
            Program_Id__c = e.Program__c,
            Contact_Id__c = e.Contact__c,
            New_Status__c = e.Status__c
        );
    }
}
```

No recursion guard is needed: `fillOpenSeats` only moves Waitlisted to Offered, which never frees a seat, so the nested trigger run publishes the Offered events and stops.

2c. Capacity increases

File: `force-app/main/default/triggers/ProgramTrigger.trigger`

```
trigger ProgramTrigger on Program__c (after update) {
    Set<Id> grew = new Set<Id>();
    for (Program__c p : Trigger.new) {
        if (p.Capacity__c > Trigger.oldMap.get(p.Id).Capacity__c) {
            grew.add(p.Id);
        }
    }
    if (!grew.isEmpty()) {
        EnrollmentService.fillOpenSeats(grew);
    }
}
```

2d. Offer expiration job

File: `force-app/main/default/classes/OfferExpirationJob.cls`

```
/**
 * Expires offers past their deadline. The trigger then offers each freed seat
 * to the next waitlisted family. Schedule hourly:
 * System.schedule('Expire enrollment offers hourly', '0 0 * * * ?', new OfferExpirationJob());
 */
public with sharing class OfferExpirationJob implements Database.Batchable<SObject>, Schedulable {
    public Database.QueryLocator start(Database.BatchableContext bc) {
        Datetime now = System.now();
        return Database.getQueryLocator([
            SELECT Id FROM Enrollment__c
            WHERE Status__c = 'Offered' AND Offer_Expires_At__c < :now
        ]);
    }

    public void execute(Database.BatchableContext bc, List<Enrollment__c> scope) {
        for (Enrollment__c e : scope) {
            e.Status__c = 'Expired';
        }
        update scope;
    }

    public void finish(Database.BatchableContext bc) {
    }

    public void execute(SchedulableContext sc) {
        Database.executeBatch(new OfferExpirationJob(), 200);
    }
}
```

Step 3: LWC and notifications

One record-page component shows capacity, offers, enrollments and the waitlist, and refreshes itself live when any enrollment changes. A platform-event-triggered Flow sends the family emails, so notification logic stays out of Apex.

3a. EnrollmentController.cls

File: `force-app/main/default/classes/EnrollmentController.cls`

```
public with sharing class EnrollmentController {
    public class Row {
        @AuraEnabled public Id id;
        @AuraEnabled public String contactName;
        @AuraEnabled public String status;
        @AuraEnabled public Integer position;
        @AuraEnabled public Datetime offerExpiresAt;
    }

    public class Summary {
        @AuraEnabled public Integer capacity;
        @AuraEnabled public Integer seatsHeld;
        @AuraEnabled public List<Row> enrolled = new List<Row>();
        @AuraEnabled public List<Row> offered = new List<Row>();
        @AuraEnabled public List<Row> waitlist = new List<Row>();
    }

    @AuraEnabled(cacheable=true)
    public static Summary getSummary(Id programId) {
        Program__c p = [SELECT Capacity__c FROM Program__c WHERE Id = :programId WITH USER_MODE];
        Summary s = new Summary();
        s.capacity = p.Capacity__c.intValue();
        Integer position = 0;
        for (Enrollment__c e : [SELECT Id, Contact__r.Name, Status__c, Offer_Expires_At__c
                                FROM Enrollment__c
                                WHERE Program__c = :programId AND Status__c IN :EnrollmentService.ACTIVE
```

```
                                WITH USER_MODE
                                ORDER BY Waitlisted_At__c ASC NULLS LAST, Enrolled_At__c ASC, Name ASC]) {
            Row r = new Row();
            r.id = e.Id;
            r.contactName = e.Contact__r.Name;
            r.status = e.Status__c;
            r.offerExpiresAt = e.Offer_Expires_At__c;
            if (e.Status__c == 'Waitlisted') {
                position++;
                r.position = position;
                s.waitlist.add(r);
            } else if (e.Status__c == 'Offered') {
                s.offered.add(r);
            } else {
                s.enrolled.add(r);
            }
        }
        s.seatsHeld = s.enrolled.size() + s.offered.size();
        return s;
    }

    @AuraEnabled
    public static void enroll(Id programId, Id contactId) {
        try {
            EnrollmentService.enroll(programId, contactId);
        } catch (EnrollmentService.EnrollmentException ex) {
            throw auraError(ex.getMessage());
        }
    }

    @AuraEnabled
    public static void changeStatus(Id enrollmentId, String newStatus) {
        try {
            EnrollmentService.changeStatus(enrollmentId, newStatus);
        } catch (EnrollmentService.EnrollmentException ex) {
            throw auraError(ex.getMessage());
        }
    }

    private static AuraHandledException auraError(String msg) {
        AuraHandledException e = new AuraHandledException(msg);
        e.setMessage(msg); // makes the message assertable in tests
        return e;
    }
}
```

3b. programEnrollment.js

File: `force-app/main/default/lwc/programEnrollment/programEnrollment.js`

```
        import { LightningElement, api, wire } from 'lwc';
        import { refreshApex } from '@salesforce/apex';
        import { ShowToastEvent } from 'lightning/platformShowToastEvent';
        import { subscribe, unsubscribe, onError } from 'lightning/empApi';
        import getSummary from '@salesforce/apex/EnrollmentController.getSummary';
        import enroll from '@salesforce/apex/EnrollmentController.enroll';
        import changeStatus from '@salesforce/apex/EnrollmentController.changeStatus';

        const CHANNEL = '/event/Enrollment_Change__e';

        export default class ProgramEnrollment extends LightningElement {
            @api recordId;
            summary;
            wiredResult;
            selectedContactId;
            subscription;

            @wire(getSummary, { programId: '$recordId' })
            wired(result) {
                this.wiredResult = result;
                if (result.data) {
                    this.summary = result.data;
                } else if (result.error) {
                    this.toast('Error', this.message(result.error), 'error');
                }
            }

            get percentFull() {
                if (!this.summary || !this.summary.capacity) {
                    return 0;
                }
                return Math.min(100, Math.round((this.summary.seatsHeld / this.summary.capacity) * 100));
            }

            get seatsLabel() {
                return this.summary ? `${this.summary.seatsHeld} of ${this.summary.capacity} seats held` : '';
```

```
    }

    get noContact() {
        return !this.selectedContactId;
    }

    get hasOffers() {
        return this.summary && this.summary.offered.length > 0;
    }

    get hasEnrolled() {
        return this.summary && this.summary.enrolled.length > 0;
    }

    get hasWaitlist() {
        return this.summary && this.summary.waitlist.length > 0;
    }

    connectedCallback() {
        subscribe(CHANNEL, -1, (msg) => {
            if (msg.data.payload.Program_Id__c === this.recordId) {
                refreshApex(this.wiredResult);
            }
        }).then((sub) => {
            this.subscription = sub;
        });
        onError((err) => {
            // eslint-disable-next-line no-console
            console.error('empApi error', JSON.stringify(err));
        });
    }

    disconnectedCallback() {
        if (this.subscription) {
            unsubscribe(this.subscription);
        }
    }

    handleContactChange(event) {
        this.selectedContactId = event.detail.recordId;
    }

    async handleEnroll() {
        try {
            await enroll({ programId: this.recordId, contactId: this.selectedContactId });
            this.toast('Saved', 'Enrollment created', 'success');
            await refreshApex(this.wiredResult);
        } catch (e) {
            this.toast('Could not enroll', this.message(e), 'error');
        }
    }

    async handleAction(event) {
        const { id, status } = event.target.dataset;
        try {
            await changeStatus({ enrollmentId: id, newStatus: status });
            await refreshApex(this.wiredResult);
        } catch (e) {
            this.toast('Update failed', this.message(e), 'error');
        }
    }

    message(e) {
        return e?.body?.message ?? 'Unexpected error';
    }

    toast(title, message, variant) {
        this.dispatchEvent(new ShowToastEvent({ title, message, variant }));
    }
}
```

3c. programEnrollment.html

File: `force-app/main/default/lwc/programEnrollment/programEnrollment.html`

```
<template>
    <lightning-card title="Enrollment" icon-name="standard:education">
        <div class="slds-p-horizontal_medium" lwc:if={summary}>
            <p class="slds-text-body_small slds-m-bottom_xx-small">{seatsLabel}</p>
            <lightning-progress-bar value={percentFull} size="large"></lightning-progress-bar>

            <div class="slds-grid slds-gutters slds-m-top_medium slds-grid_vertical-align-end">
                <div class="slds-col">
                    <lightning-record-picker
                        label="Contact"
                        placeholder="Search contacts..."
```

```
                                object-api-name="Contact"
                                onchange={handleContactChange}>
                            </lightning-record-picker>
                        </div>
                        <div class="slds-col slds-grow-none">
                            <lightning-button label="Enroll" variant="brand" disabled={noContact} onclick={handleEnroll}></lightning-button>
                        </div>
                    </div>

                    <h3 class="slds-text-heading_small slds-m-top_medium">Offers pending</h3>
                    <template lwc:if={hasOffers}>
                        <template for:each={summary.offered} for:item="row">
                            <div key={row.id} class="slds-grid slds-grid_vertical-align-center slds-p-vertical_xx-small">
                                <span class="slds-col">
                                    {row.contactName}, expires
                                    <lightning-formatted-date-time value={row.offerExpiresAt} month="short" day="numeric"
                                        hour="2-digit" minute="2-digit"></lightning-formatted-date-time>
                                </span>
                                <lightning-button label="Accept" data-id={row.id} data-status="Enrolled" onclick={handleAction}
                                    class="slds-m-left_x-small"></lightning-button>
                                <lightning-button label="Decline" data-id={row.id} data-status="Declined" onclick={handleAction}
                                    class="slds-m-left_x-small"></lightning-button>
                            </div>
                        </template>
                    </template>
                    <p lwc:else class="slds-text-color_weak">No pending offers.</p>

                    <h3 class="slds-text-heading_small slds-m-top_medium">Enrolled</h3>
                    <template lwc:if={hasEnrolled}>
                        <template for:each={summary.enrolled} for:item="row">
                            <div key={row.id} class="slds-grid slds-grid_vertical-align-center slds-p-vertical_xx-small">
                                <span class="slds-col">{row.contactName}</span>
                                <lightning-button label="Withdraw" data-id={row.id} data-status="Withdrawn" onclick={handleAction}></lightni
button>
                            </div>
                        </template>
                    </template>
                    <p lwc:else class="slds-text-color_weak">Nobody enrolled yet.</p>

                    <h3 class="slds-text-heading_small slds-m-top_medium">Waitlist</h3>
                    <template lwc:if={hasWaitlist}>
                        <template for:each={summary.waitlist} for:item="row">
                            <div key={row.id} class="slds-grid slds-grid_vertical-align-center slds-p-vertical_xx-small">
                                <span class="slds-col">#{row.position} {row.contactName}</span>
                                <lightning-button label="Withdraw" data-id={row.id} data-status="Withdrawn" onclick={handleAction}></lightni
button>
                            </div>
                        </template>
                    </template>
                    <p lwc:else class="slds-text-color_weak slds-m-bottom_small">Nobody is waiting.</p>
                </div>
            </lightning-card>
        </template>
```

Place it on the page: open any Program record, click the gear, Edit Page, drag **Program Enrollment** onto the layout, Save, and Activate as the org default.

3d. Email notifications with a platform-event-triggered Flow (manual)

1. Setup, Flows, New Flow, Platform Event-Triggered Flow. Choose `Enrollment_Change__e`.
2. Get Records: the Contact where Id equals `{!$Record.Contact_Id__c}`, and the Program__c where Id equals `{!$Record.Program_Id__c}`.
3. Decision on `{!$Record.New_Status__c}` with outcomes Enrolled, Waitlisted and Offered.
4. Each outcome: a Text Template for the body, then the Send Email action to the Contact's email. The Offered email should state the expiry time and who to contact.
5. Setup, Deliverability: set access level to All email. The seed script's `emailDomain` variable lets you point demo contacts at your own inbox.

Why a platform event here: with Publish After Commit, emails go out only if the transaction commits. A rolled-back enrollment never sends a false confirmation.

Step 4: Tests, demo script, talking points

The shipped test suite asserts behavior, not just lines executed. Interviewers who open the repo usually read the test class first.

Test class	Test method	Proves
EnrollmentServiceTest	enrollUnderCapacityIsEnrolled	Open seat means Enrolled

	enrollAtCapacityIsWaitlisted	Full program means Waitlisted
	duplicateEnrollmentIsRejected	Second active enrollment throws
	invalidTransitionIsRejected	Waitlisted to Enrolled directly throws
	withdrawalOffersSeatToOldestWaitlisted	The promotion loop
	acceptingOfferEnrolls	Offer acceptance stamps Enrolled_At
	capacityIncreaseOffersMultipleSeats	Raising capacity 1 to 3 offers 2 families
	deletingEnrollmentFreesSeat	After-delete path promotes the next family
	expiredOfferPromotesNextFamily	Batch expires past-due offers, next family offered
	bulkWithdrawal200	200 withdrawals in one DML stay within limits
	statusChangePublishesEvent	A platform event is published per change
	scheduleJobRegistersCron	The job schedules correctly
EnrollmentControllerTest	summaryListsQueueInOrder, enrollAndChangeStatusSucceed, controllerWrapsErrors	LWC contract and AuraHandledException messages
AgentActionsTest	findProgramsWithOpenings, waitlistSummaryAndAmbiguity, updateStatusOutcomes	Every agent action outcome, including ambiguity and rule violations

Test data factory

File: `force-app/main/default/classes/TestDataFactory.cls`

```
@IsTest
public class TestDataFactory {
    public static Program__c program(Integer capacity) {
        return program('Test Program', capacity);
    }

    public static Program__c program(String name, Integer capacity) {
        Program__c p = new Program__c(Name = name, Capacity__c = capacity, Offer_Window_Hours__c = 48);
        insert p;
        return p;
    }

    public static List<Contact> contacts(Integer n) {
        List<Contact> cs = new List<Contact>();
        for (Integer i = 0; i < n; i++) {
            String suffix = String.valueOf(i).leftPad(3, '0');
            cs.add(new Contact(LastName = 'Test Family ' + suffix, Email = 'family' + suffix + '@example.com'));
        }
        insert cs;
        return cs;
    }

    /** Inserts enrollments directly (bypassing the service) with distinct, ordered timestamps. */
    public static List<Enrollment__c> enrollments(Id programId, List<Contact> cs, String status) {
        Datetime base = System.now().addHours(-1);
        List<Enrollment__c> rows = new List<Enrollment__c>();
        for (Integer i = 0; i < cs.size(); i++) {
            Enrollment__c e = new Enrollment__c(Program__c = programId, Contact__c = cs[i].Id, Status__c = status);
            if (status == 'Waitlisted') {
                e.Waitlisted_At__c = base.addSeconds(i);
            } else if (status == 'Enrolled') {
                e.Enrolled_At__c = base.addSeconds(i);
            } else if (status == 'Offered') {
                e.Offered_At__c = base;
                e.Offer_Expires_At__c = base.addHours(48);
            }
            rows.add(e);
        }
        insert rows;
        return rows;
    }

    public static Map<Id, Enrollment__c> reload() {
        return new Map<Id, Enrollment__c>([
            SELECT Id, Status__c, Offer_Expires_At__c, Enrolled_At__c FROM Enrollment__c
        ]);
    }
```

```
}
```

Core service tests

File: `force-app/main/default/classes/EnrollmentServiceTest.cls`

```
@IsTest
private class EnrollmentServiceTest {
    @IsTest
    static void enrollUnderCapacityIsEnrolled() {
        Program__c p = TestDataFactory.program(2);
        List<Contact> cs = TestDataFactory.contacts(1);

        Test.startTest();
        Enrollment__c e = EnrollmentService.enroll(p.Id, cs[0].Id);
        Test.stopTest();

        Assert.areEqual('Enrolled', TestDataFactory.reload().get(e.Id).Status__c);
    }

    @IsTest
    static void enrollAtCapacityIsWaitlisted() {
        Program__c p = TestDataFactory.program(1);
        List<Contact> cs = TestDataFactory.contacts(2);
        EnrollmentService.enroll(p.Id, cs[0].Id);

        Test.startTest();
        Enrollment__c second = EnrollmentService.enroll(p.Id, cs[1].Id);
        Test.stopTest();

        Assert.areEqual('Waitlisted', TestDataFactory.reload().get(second.Id).Status__c);
    }

    @IsTest
    static void duplicateEnrollmentIsRejected() {
        Program__c p = TestDataFactory.program(5);
        List<Contact> cs = TestDataFactory.contacts(1);
        EnrollmentService.enroll(p.Id, cs[0].Id);

        try {
            EnrollmentService.enroll(p.Id, cs[0].Id);
            Assert.fail('Expected a duplicate enrollment error');
        } catch (EnrollmentService.EnrollmentException ex) {
            Assert.isTrue(ex.getMessage().contains('already'), ex.getMessage());
        }
    }

    @IsTest
    static void invalidTransitionIsRejected() {
        Program__c p = TestDataFactory.program(1);
        List<Contact> cs = TestDataFactory.contacts(1);
        Enrollment__c w = TestDataFactory.enrollments(p.Id, cs, 'Waitlisted')[0];

        try {
            EnrollmentService.changeStatus(w.Id, 'Enrolled');
            Assert.fail('Waitlisted to Enrolled should be rejected');
        } catch (EnrollmentService.EnrollmentException ex) {
            Assert.isTrue(ex.getMessage().contains('Cannot move'), ex.getMessage());
        }
    }

    @IsTest
    static void withdrawalOffersSeatToOldestWaitlisted() {
        Program__c p = TestDataFactory.program(1);
        List<Contact> cs = TestDataFactory.contacts(3);
        Enrollment__c firstEnrollment = TestDataFactory.enrollments(p.Id, new List<Contact>{ cs[0] }, 'Enrolled')[0];
        List<Enrollment__c> waiting = TestDataFactory.enrollments(p.Id, new List<Contact>{ cs[1], cs[2] }, 'Waitlisted');

        Test.startTest();
        EnrollmentService.changeStatus(firstEnrollment.Id, 'Withdrawn');
        Test.stopTest();

        Map<Id, Enrollment__c> result = TestDataFactory.reload();
        Assert.areEqual('Withdrawn', result.get(firstEnrollment.Id).Status__c);
        Assert.areEqual('Offered', result.get(waiting[0].Id).Status__c, 'Oldest waitlisted family gets the seat');
        Assert.isNotNull(result.get(waiting[0].Id).Offer_Expires_At__c, 'Offer has an expiry');
        Assert.areEqual('Waitlisted', result.get(waiting[1].Id).Status__c, 'Next family keeps waiting');
    }

    @IsTest
    static void acceptingOfferEnrolls() {
        Program__c p = TestDataFactory.program(1);
        List<Contact> cs = TestDataFactory.contacts(1);
        Enrollment__c offer = TestDataFactory.enrollments(p.Id, cs, 'Offered')[0];

        Test.startTest();
```

```
        EnrollmentService.changeStatus(offer.Id, 'Enrolled');
        Test.stopTest();

        Enrollment__c result = TestDataFactory.reload().get(offer.Id);
        Assert.areEqual('Enrolled', result.Status__c);
        Assert.isNotNull(result.Enrolled_At__c);
    }

    @IsTest
    static void capacityIncreaseOffersMultipleSeats() {
        Program__c p = TestDataFactory.program(1);
        List<Contact> cs = TestDataFactory.contacts(4);
        TestDataFactory.enrollments(p.Id, new List<Contact>{ cs[0] }, 'Enrolled');
        List<Enrollment__c> waiting = TestDataFactory.enrollments(p.Id, new List<Contact>{ cs[1], cs[2], cs[3] }, 'Waitlisted');

        Test.startTest();
        p.Capacity__c = 3;
        update p;
        Test.stopTest();

        Map<Id, Enrollment__c> result = TestDataFactory.reload();
        Assert.areEqual('Offered', result.get(waiting[0].Id).Status__c);
        Assert.areEqual('Offered', result.get(waiting[1].Id).Status__c);
        Assert.areEqual('Waitlisted', result.get(waiting[2].Id).Status__c);
    }

    @IsTest
    static void deletingEnrollmentFreesSeat() {
        Program__c p = TestDataFactory.program(1);
        List<Contact> cs = TestDataFactory.contacts(2);
        Enrollment__c firstEnrollment = TestDataFactory.enrollments(p.Id, new List<Contact>{ cs[0] }, 'Enrolled')[0];
        Enrollment__c nextFamily = TestDataFactory.enrollments(p.Id, new List<Contact>{ cs[1] }, 'Waitlisted')[0];

        Test.startTest();
        delete firstEnrollment;
        Test.stopTest();

        Assert.areEqual('Offered', TestDataFactory.reload().get(nextFamily.Id).Status__c);
    }

    @IsTest
    static void expiredOfferPromotesNextFamily() {
        Program__c p = TestDataFactory.program(1);
        List<Contact> cs = TestDataFactory.contacts(2);
        Enrollment__c offer = TestDataFactory.enrollments(p.Id, new List<Contact>{ cs[0] }, 'Offered')[0];
        Enrollment__c nextFamily = TestDataFactory.enrollments(p.Id, new List<Contact>{ cs[1] }, 'Waitlisted')[0];
        offer.Offer_Expires_At__c = System.now().addHours(-1);
        update offer;

        Test.startTest();
        Database.executeBatch(new OfferExpirationJob(), 200);
        Test.stopTest();

        Map<Id, Enrollment__c> result = TestDataFactory.reload();
        Assert.areEqual('Expired', result.get(offer.Id).Status__c);
        Assert.areEqual('Offered', result.get(nextFamily.Id).Status__c);
    }

    @IsTest
    static void bulkWithdrawal200() {
        Program__c p = TestDataFactory.program(200);
        List<Contact> cs = TestDataFactory.contacts(250);
        List<Contact> enrolledContacts = new List<Contact>();
        List<Contact> waitingContacts = new List<Contact>();
        for (Integer i = 0; i < cs.size(); i++) {
            if (i < 200) {
                enrolledContacts.add(cs[i]);
            } else {
                waitingContacts.add(cs[i]);
            }
        }
        List<Enrollment__c> enrolled = TestDataFactory.enrollments(p.Id, enrolledContacts, 'Enrolled');
        TestDataFactory.enrollments(p.Id, waitingContacts, 'Waitlisted');

        Test.startTest();
        for (Enrollment__c e : enrolled) {
            e.Status__c = 'Withdrawn';
        }
        update enrolled;
        Test.stopTest();

        Integer offered = [SELECT COUNT() FROM Enrollment__c WHERE Status__c = 'Offered'];
        Assert.areEqual(50, offered, 'All 50 waitlisted families are offered seats in one transaction');
    }

    @IsTest
    static void statusChangePublishesEvent() {
        Program__c p = TestDataFactory.program(1);
        List<Contact> cs = TestDataFactory.contacts(1);
```

```
        Integer eventsBefore = EnrollmentTriggerHandler.eventsPublished;

        Test.startTest();
        EnrollmentService.enroll(p.Id, cs[0].Id);
        Test.getEventBus().deliver();
        Test.stopTest();

        Assert.areEqual(eventsBefore + 1, EnrollmentTriggerHandler.eventsPublished);
    }

    @IsTest
    static void scheduleJobRegistersCron() {
        Test.startTest();
        String jobId = System.schedule('Offer expiry test', '0 0 * * * ?', new OfferExpirationJob());
        Test.stopTest();

        Assert.areEqual(1, [SELECT COUNT() FROM CronTrigger WHERE Id = :jobId]);
    }
}
```

Demo script (2 to 3 minutes)

1. Open **Infant Care** (capacity 4, from the seed data). Point out the capacity bar and Maria Lopez's pending offer.
2. Open **Pre-K Morning**: 3 enrolled, 3 waiting. Enroll a new contact and show them land at the end of the waitlist.
3. Try to enroll the same family twice to show the error toast.
4. With the page open in two browser tabs, withdraw an enrolled family. Both tabs update live, and waitlist #1 becomes Offered with an expiry time.
5. Show the offer email arriving in your inbox.
6. Set that offer's expiry to the past and run `Database.executeBatch(new OfferExpirationJob());`. The offer expires and the next family is offered.
7. Raise capacity and watch the waitlist clear.
8. End on the test run and coverage in VS Code.

Talking points

- **Concurrency:** `FOR UPDATE` on the program row prevents two coordinators from overbooking the last seat. Be ready to explain the race condition it prevents.
- **Single service layer:** the trigger, LWC, batch job and the Agentforce agent all call the same methods.
- **Explicit state machine:** allowed transitions are data, not scattered if statements, so invalid moves fail loudly.
- **Bulk safety:** `fillOpenSeats` uses three queries and one DML regardless of volume.
- **Transactional notifications:** Publish After Commit means no email for a rolled-back change.
- **Security:** `with sharing`, `WITH USER_MODE` queries, and a dedicated permission set.
- **Nonprofit Cloud mapping:** Program__c maps to Nonprofit Cloud's Program object and Enrollment__c to Program Enrollment. Explain what you would reuse versus rebuild on a greenfield Nonprofit Cloud org.
- **What you would add next:** an Experience Cloud page where families accept offers themselves, and priority rules (for example siblings first) stored in custom metadata.

Project 2: Agentforce Enrollment Assistant

Step 1: Org setup and data

An internal “Enrollment Assistant” for program coordinators, built on Project 1. An employee-facing agent needs no identity verification or chat channel setup, so you spend your time on actions and instructions, which is what interviewers ask about.

Architecture: Coordinator (Agentforce panel in Lightning) → Enrollment Assistant (one topic: Enrollment Operations) → four actions. Three are Apex that call `EnrollmentService`, so the agent cannot break a rule the UI enforces. The fourth is a prompt template that reads record fields to draft a message.

1. Use the same Agentforce-enabled org from Part 0, with the package deployed and seed data loaded.
2. In Setup, confirm Einstein generative AI and Agentforce Agents are turned on (orgs of this type usually have them on already).
3. Setup, Agentforce Agents, New Agent. Choose the employee-facing agent type and name it **Enrollment Assistant**. Remove default topics you will not use.
4. Employee agents act as the signed-in user. The `Enrollment_Coordinator` permission set already grants Apex class access to the three action classes; missing class access is the most common reason an action silently fails.

LLM usage in the free org is rate-limited, so test in short, focused sessions.

Step 2: Apex actions and prompt template

The agent’s reasoning engine picks actions by reading their labels and descriptions, so they are written as carefully as the code. Each action returns a plain-text summary, and failures come back as messages rather than exceptions so the agent can explain them.

2a. FindProgramsWithOpeningsAction.cls

File: `force-app/main/default/classes/FindProgramsWithOpeningsAction.cls`

```
public with sharing class FindProgramsWithOpeningsAction {
    public class Request {
        @InvocableVariable(label='Minimum Seats' description='Only return programs with at least this many open seats. Defaults to 1.')
        public Integer minSeats;
    }

    public class Response {
        @InvocableVariable(label='Programs Summary' description='One line per program: name, open seats, families waiting.')
        public String summary;
    }

    @InvocableMethod(label='Find Programs With Openings'
        description='Lists programs that currently have open seats, with the number of open seats and waitlisted families.')
    public static List<Response> run(List<Request> requests) {
        Map<Id, Integer> waiting = new Map<Id, Integer>();
        for (AggregateResult ar : [SELECT Program__c programId, COUNT(Id) total FROM Enrollment__c
                                   WHERE Status__c = 'Waitlisted' WITH USER_MODE GROUP BY Program__c]) {
            waiting.put((Id) ar.get('programId'), (Integer) ar.get('total'));
        }
        List<Program__c> programs = [SELECT Id, Name, Capacity__c, Seats_Held__c FROM Program__c
                                     WITH USER_MODE ORDER BY Name];

        List<Response> out = new List<Response>();
        for (Request req : requests) {
            Integer minSeats = (req == null || req.minSeats == null || req.minSeats < 1) ? 1 : req.minSeats;
            List<String> lines = new List<String>();
            for (Program__c p : programs) {
                Integer heldSeats = p.Seats_Held__c == null ? 0 : p.Seats_Held__c.intValue();
                Integer openSeats = p.Capacity__c.intValue() - heldSeats;
                if (openSeats >= minSeats) {
                    Integer w = waiting.containsKey(p.Id) ? waiting.get(p.Id) : 0;
                    lines.add(p.Name + ': ' + openSeats + ' open seat(s), ' + w + ' waiting');
                }
            }
            Response r = new Response();
            r.summary = lines.isEmpty()
                ? 'No programs currently have ' + minSeats + ' or more open seats.'
                : String.join(lines, '\n');
            out.add(r);
        }
        return out;
    }
}
```

2b. GetProgramWaitlistAction.cls

File: `force-app/main/default/classes/GetProgramWaitlistAction.cls`

```
public with sharing class GetProgramWaitlistAction {
    public class Request {
        @InvocableVariable(label='Program Name' description='Full or partial program name, for example Pre-K Morning' required=true)
        public String programName;
    }

    public class Response {
        @InvocableVariable(label='Waitlist' description='Seats held, pending offers with expiry, and the waitlist in queue order. Each
line includes the Enrollment Id.')
        public String summary;
    }

    @InvocableMethod(label='Get Program Waitlist'
        description='Returns seats held versus capacity, pending offers with expiry times, and the ordered waitlist for one program.')
    public static List<Response> run(List<Request> requests) {
        List<Response> out = new List<Response>();
        // Agents send one request at a time; bulkify the lookup before reusing this from Flow at scale
        for (Request req : requests) {
            out.add(summarize(req.programName));
        }
        return out;
    }

    private static Response summarize(String programName) {
        Response r = new Response();
        String pattern = '%' + programName + '%';
        List<Program__c> matches = [SELECT Id, Name FROM Program__c WHERE Name LIKE :pattern WITH USER_MODE LIMIT 5];
        if (matches.size() != 1) {
            List<String> names = new List<String>();
            for (Program__c p : matches) {
                names.add(p.Name);
            }
            r.summary = matches.isEmpty()
                ? 'No program matches "' + programName + '".'
                : 'More than one program matches; ask which one: ' + String.join(names, ', ');
            return r;
        }

        EnrollmentController.Summary s = EnrollmentController.getSummary(matches[0].Id);
        List<String> lines = new List<String>{
            matches[0].Name + ': ' + s.seatsHeld + ' of ' + s.capacity + ' seats held.'
        };
        for (EnrollmentController.Row o : s.offered) {
            String expiry = o.offerExpiresAt == null ? 'no expiry set' : o.offerExpiresAt.format();
            lines.add('Offer pending: ' + o.contactName + ', expires ' + expiry + ' (Enrollment Id ' + o.id + ')');
        }
        for (EnrollmentController.Row w : s.waitlist) {
            lines.add('#' + w.position + ' ' + w.contactName + ' (Enrollment Id ' + w.id + ')');
        }
        if (s.waitlist.isEmpty()) {
            lines.add('Nobody is waiting.');
        }
        r.summary = String.join(lines, '\n');
        return r;
    }
}
```

Reusing `EnrollmentController.getSummary` means the agent and the LWC compute waitlist positions the same way.

2c. UpdateEnrollmentStatusAction.cls

File: `force-app/main/default/classes/UpdateEnrollmentStatusAction.cls`

```
public with sharing class UpdateEnrollmentStatusAction {
    public class Request {
        @InvocableVariable(label='Contact Name' description='Name of the family contact on the enrollment' required=true)
        public String contactName;
        @InvocableVariable(label='Program Name' description='Full or partial program name' required=true)
        public String programName;
        @InvocableVariable(label='New Status' description='One of: Enrolled (accept an offer), Declined (decline an offer), Withdrawn
(leave the program or waitlist)' required=true)
        public String newStatus;
    }

    public class Response {
        @InvocableVariable(label='Result' description='What changed, or why nothing changed')
        public String message;
    }

    @InvocableMethod(label='Update Enrollment Status'
```

```
        description='Accepts or declines a pending offer, or withdraws a family from a program or waitlist. A freed seat is offered to
the next waitlisted family automatically.')
    public static List<Response> run(List<Request> requests) {
        List<Response> out = new List<Response>();
        for (Request req : requests) {
            Response r = new Response();
            String contactPattern = '%' + req.contactName + '%';
            String programPattern = '%' + req.programName + '%';
            try {
                List<Enrollment__c> matches = [
                    SELECT Id, Status__c, Contact__r.Name, Program__r.Name FROM Enrollment__c
                    WHERE Contact__r.Name LIKE :contactPattern
                    AND Program__r.Name LIKE :programPattern
                    AND Status__c IN :EnrollmentService.ACTIVE
                    WITH USER_MODE LIMIT 5];
                if (matches.isEmpty()) {
                    r.message = 'No active enrollment found for ' + req.contactName + ' in ' + req.programName + '.';
                } else if (matches.size() > 1) {
                    r.message = 'Several enrollments match; ask the user for the full contact and program name.';
                } else {
                    Enrollment__c e = matches[0];
                    String priorStatus = e.Status__c;
                    EnrollmentService.changeStatus(e.Id, req.newStatus);
                    r.message = e.Contact__r.Name + ' moved from ' + priorStatus + ' to ' + req.newStatus
                        + ' in ' + e.Program__r.Name + ' (Enrollment Id ' + e.Id + ').';
                }
            } catch (EnrollmentService.EnrollmentException ex) {
                r.message = 'Not changed: ' + ex.getMessage();
            }
            out.add(r);
        }
        return out;
    }
}
```

The agent cannot make a move the UI forbids: an attempt to jump a family from Waitlisted straight to Enrolled comes back as “Not changed” with the reason from the transition map.

2d. Prompt template: Draft Family Update (manual)

1. Setup, Prompt Builder, New Prompt Template. Type: Flex. Name: Draft Family Update.
2. Add one input: Object, Enrollment__c, API name `Enrollment`.
3. Write the template below, inserting fields with the merge field picker (it produces the exact tokens your org expects; the tokens shown are illustrative).
4. Preview it against an Offered and a Waitlisted record, then Save and Activate.

```
You write on behalf of a childcare nonprofit's enrollment team.
Write a short, warm email (under 120 words) to {!$Input:Enrollment.Contact__r.Name}
about their enrollment in {!$Input:Enrollment.Program__r.Name}.
Current status: {!$Input:Enrollment.Status__c}
Offer expires: {!$Input:Enrollment.Offer_Expires_At__c}

Rules:
- If the status is Offered, state the expiry time clearly and ask them to reply to accept.
- If the status is Waitlisted, thank them and say we will contact them when a seat
  opens. Do not promise a date.
- Do not invent fees, dates, or policies that are not in this prompt.
- Sign off as "The Enrollment Team".
```

The agent passes the Enrollment Id from the waitlist or update output into this template, which is why those actions include Ids in their text.

Agent action tests

File: `force-app/main/default/classes/AgentActionsTest.cls`

```
@IsTest
private class AgentActionsTest {
    @IsTest
    static void findProgramsWithOpenings() {
        Program__c openProgram = TestDataFactory.program('Infant Care', 3);
        Program__c fullProgram = TestDataFactory.program('Toddler Play', 1);
        List<Contact> cs = TestDataFactory.contacts(2);
        TestDataFactory.enrollments(openProgram.Id, new List<Contact>{ cs[0] }, 'Enrolled');
        TestDataFactory.enrollments(fullProgram.Id, new List<Contact>{ cs[1] }, 'Enrolled');

        FindProgramsWithOpeningsAction.Request anySeats = new FindProgramsWithOpeningsAction.Request();
        FindProgramsWithOpeningsAction.Request many = new FindProgramsWithOpeningsAction.Request();
        many.minSeats = 5;
```

```
        Test.startTest();
        List<FindProgramsWithOpeningsAction.Response> out =
            FindProgramsWithOpeningsAction.run(new List<FindProgramsWithOpeningsAction.Request>{ anySeats, many });
        Test.stopTest();

        Assert.isTrue(out[0].summary.contains('Infant Care: 2 open seat(s)'), out[0].summary);
        Assert.isFalse(out[0].summary.contains('Toddler Play'), out[0].summary);
        Assert.isTrue(out[1].summary.startsWith('No programs'), out[1].summary);
    }

    @IsTest
    static void waitlistSummaryAndAmbiguity() {
        Program__c morning = TestDataFactory.program('Pre-K Morning', 1);
        TestDataFactory.program('Pre-K Afternoon', 5);
        List<Contact> cs = TestDataFactory.contacts(3);
        TestDataFactory.enrollments(morning.Id, new List<Contact>{ cs[0] }, 'Offered');
        TestDataFactory.enrollments(morning.Id, new List<Contact>{ cs[1], cs[2] }, 'Waitlisted');

        List<GetProgramWaitlistAction.Request> reqs = new List<GetProgramWaitlistAction.Request>();
        for (String name : new List<String>{ 'Pre-K Morning', 'Pre-K', 'Swimming' }) {
            GetProgramWaitlistAction.Request r = new GetProgramWaitlistAction.Request();
            r.programName = name;
            reqs.add(r);
        }

        Test.startTest();
        List<GetProgramWaitlistAction.Response> out = GetProgramWaitlistAction.run(reqs);
        Test.stopTest();

        Assert.isTrue(out[0].summary.contains('#1 Test Family 001'), out[0].summary);
        Assert.isTrue(out[0].summary.contains('Offer pending: Test Family 000'), out[0].summary);
        Assert.isTrue(out[1].summary.startsWith('More than one program matches'), out[1].summary);
        Assert.isTrue(out[2].summary.startsWith('No program matches'), out[2].summary);
    }

    @IsTest
    static void updateStatusOutcomes() {
        Program__c p = TestDataFactory.program('Infant Care', 1);
        List<Contact> cs = TestDataFactory.contacts(3);
        TestDataFactory.enrollments(p.Id, new List<Contact>{ cs[0] }, 'Offered');
        TestDataFactory.enrollments(p.Id, new List<Contact>{ cs[1], cs[2] }, 'Waitlisted');

        List<UpdateEnrollmentStatusAction.Request> reqs = new List<UpdateEnrollmentStatusAction.Request>{
            req('Test Family 000', 'Infant Care', 'Enrolled'),   // accept offer
            req('Test Family 001', 'Infant Care', 'Enrolled'),   // invalid: waitlisted straight to enrolled
            req('Nobody Here', 'Infant Care', 'Withdrawn'),      // not found
            req('Test Family 00', 'Infant Care', 'Withdrawn')    // ambiguous
        };

        Test.startTest();
        List<UpdateEnrollmentStatusAction.Response> out = UpdateEnrollmentStatusAction.run(reqs);
        Test.stopTest();

        Assert.isTrue(out[0].message.contains('moved from Offered to Enrolled'), out[0].message);
        Assert.isTrue(out[1].message.startsWith('Not changed'), out[1].message);
        Assert.isTrue(out[2].message.startsWith('No active enrollment'), out[2].message);
        Assert.isTrue(out[3].message.startsWith('Several enrollments match'), out[3].message);
    }

    private static UpdateEnrollmentStatusAction.Request req(String contactName, String programName, String status) {
        UpdateEnrollmentStatusAction.Request r = new UpdateEnrollmentStatusAction.Request();
        r.contactName = contactName;
        r.programName = programName;
        r.newStatus = status;
        return r;
    }
}
```

Step 3: Topic, instructions, testing, demo, talking points (manual)

One focused topic with explicit instructions beats several vague ones. Rules live in Apex; the instructions tell the agent when to ask, when to confirm, and how to chain actions.

3a. Create the topic

- **Name:** Enrollment Operations
- **Classification description:** Questions and tasks about program capacity, open seats, waitlists, enrollment offers, and enrollment status changes for families.
- **Scope:** Help program coordinators check capacity and waitlists, update enrollment status, and draft family communications. Do not handle billing, staff scheduling, or anything outside program enrollment.

Instructions (enter each as a separate instruction):

1. Before changing any enrollment, restate the family, program and new status, and wait for the coordinator to confirm.
2. If an action says more than one program or enrollment matches, ask a clarifying question instead of guessing.
3. For questions about who is next or where a family stands, use Get Program Waitlist and report positions in order.
4. To accept an offer use status Enrolled, to decline use Declined, to remove a family use Withdrawn. Never use any other status.
5. After a Declined or Withdrawn update, run Get Program Waitlist for that program and tell the coordinator who was offered the freed seat.
6. To draft a message, pass the Enrollment Id from the most recent action output into Draft Family Update. Present the draft and never say it was sent.
7. If a result starts with “Not changed”, explain the reason in plain language.

3b. Add the actions

1. In the topic, add each Apex class as a new agent action (reference type Apex, invocable method). Keep the generated instructions, then tighten them if tests show the wrong action being picked.
2. Mark all inputs as required except Minimum Seats.
3. For Update Enrollment Status, turn on the user confirmation setting if your org shows it; instruction 1 covers you either way.
4. Add Draft Family Update as a prompt template action and set its output to show in the conversation.
5. Activate the agent, then open the Agentforce panel from any Lightning page to test as a real user.

3c. Test plan (matches the seed data)

Run these in Agent Builder’s conversation preview and check the plan trace for the topic, action and inputs chosen. Then save them as a batch test in Testing Center so you can rerun them after every instruction change.

Utterance	Expected action(s)	Expected result
Which programs have space right now?	Find Programs With Openings	Pre-K Afternoon, 1 open seat
Who’s next on the Pre-K Morning waitlist?	Get Program Waitlist	Priya Patel is #1
Maria Lopez wants to accept her Infant Care offer	Confirm, then Update (Enrolled)	Status becomes Enrolled
Withdraw Wei Chen from Pre-K Morning	Confirm, Update (Withdrawn), Get Program Waitlist	Priya Patel is offered the seat
Enroll Noah Kim now and skip the waitlist	Update returns Not changed	Agent explains the waitlist rule
Draft an email to the family that was just offered	Draft Family Update	Draft shown, not claimed as sent
Show me the Pre-K waitlist	Get Program Waitlist	Agent asks Morning or Afternoon
What’s the weather tomorrow?	None	Polite off-topic reply

Keep a short changelog in the README of instruction changes and which failing test each one fixed; it makes a great interview story.

3d. Demo script (2 to 3 minutes)

1. Put the Program record page (with the Project 1 component) beside the Agentforce panel.
2. Ask which programs have openings, then who is next on the Pre-K Morning waitlist.
3. Ask the agent to withdraw Wei Chen. It asks for confirmation; confirm it.
4. The LWC updates live through the platform event while the agent reports that Priya Patel was offered the seat. This moment ties both projects together.
5. Ask for a draft email to Priya’s family.
6. Ask it to move waitlist #2 straight to Enrolled and show the refusal with the reason.
7. Close on the plan trace in Agent Builder to show how it chose each action.

3e. Talking points

- **Deterministic rules, probabilistic reasoning:** the LLM chooses actions, but capacity, ordering and transitions are enforced in Apex, so a bad plan cannot corrupt data.
- **Descriptions are the contract:** action labels and descriptions are how the reasoning engine selects tools, and you tuned them against a repeatable test set.
- **Errors as messages:** ambiguity and rule violations return readable text, so the agent can ask or explain instead of failing.
- **Security:** employee agents run as the signed-in user, and `WITH USER_MODE` queries mean the agent never sees more than the

coordinator can.

- **Human in the loop:** confirmation before every write, and drafts are never sent automatically.
- **Reuse:** the UI, batch job and agent share one service layer.
- **Next steps:** a customer-facing service agent for families on Experience Cloud with identity verification, and grounding on a program handbook for policy questions.

Appendix: Seed data at a glance

Program	Capacity	Enrolled	Offered	Waitlisted (in order)
Pre-K Morning	3	Wei Chen, Aisha Khan, Liam Walsh	none	Priya Patel, Noah Kim, Sofia Rossi
Pre-K Afternoon	5	Ethan Brooks, Mia Nguyen, Lucas Silva, Zara Ali	none	none (1 open seat)
Infant Care	4	Olivia Reed, James Park, Ava Johnson	Maria Lopez	Hana Sato, Leo Novak
Toddler Play	6	Emma Davis, Omar Haddad, Chloe Martin, Ryan Cooper, Nia Brown, Jack Wilson	none	Isla Moore

Mateo Garcia (Infant Care) is seeded as Withdrawn, which is what created Maria Lopez's offer. Run `scripts/apex/reset.apex` then `seed.apex` to restore this state before each recording.